UNIMORE
UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

AImage Lab

Laurea Magistrale in Ingegneria Informatica
a.a. 2023-2024

Scalable AI

Efficient Transformers

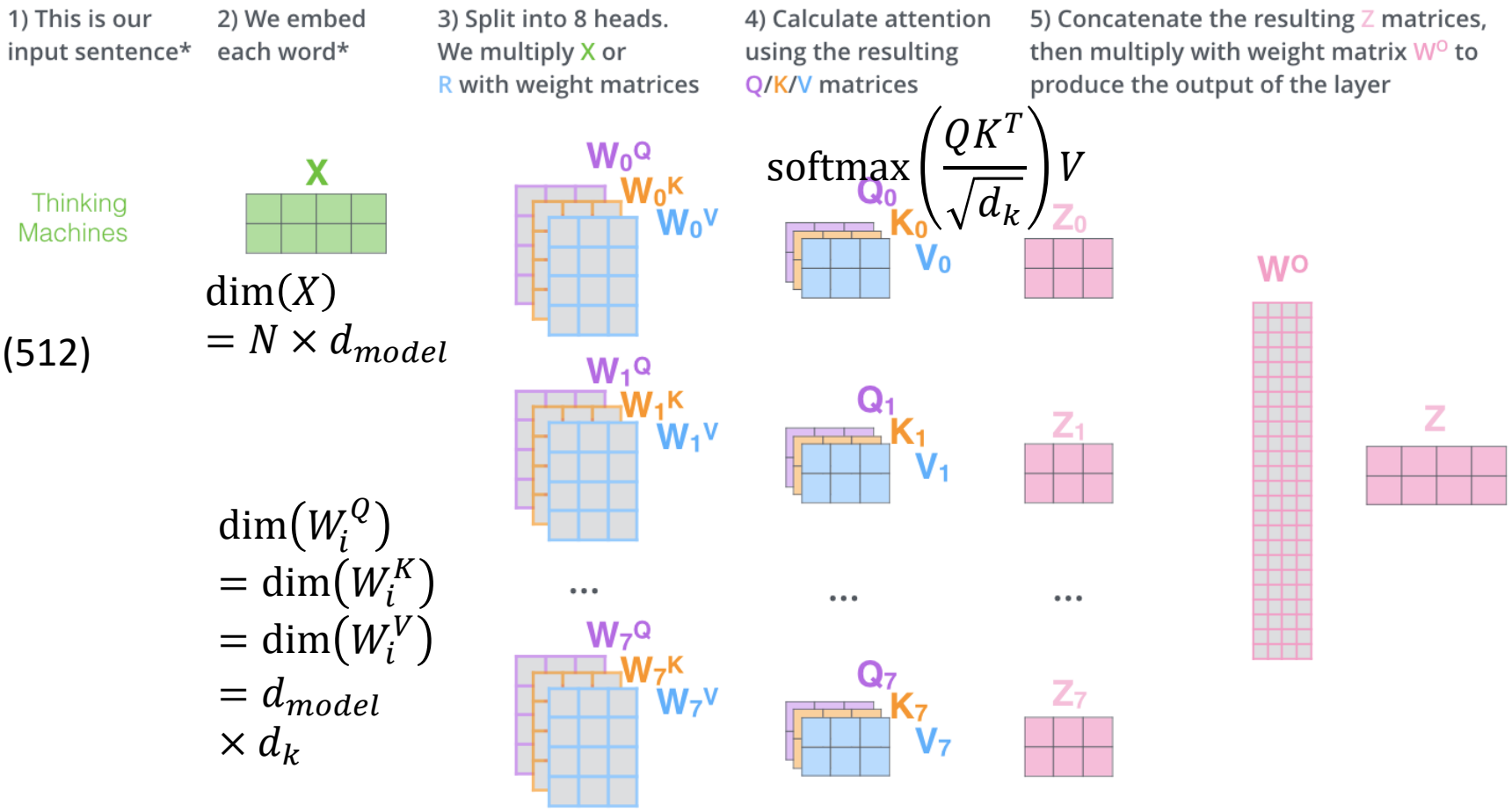
What we are covering…

UNIMORE
UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

- Quick review of transformer – attention mechanism
- Time and space complexity of traditional transformer
- Proposed solutions
  - Sparse Transformers
  - Linformer
  - Linearized Attention
  - Longformer
  - …

Quick Review of Attention Mechanism

$N :=$ sequence length
$h :=$ number of heads (8)
$d_{model} :=$ embedding size (512)
$d_k := \frac{d_{model}}{h}$ (64)



Credit: https://jalammar.github.io/illustrated-transformer/

Computational and Space Complexity

- The attention function:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- $\dim(QK^T) = N^2 \rightarrow O(N^2 d_k)$ time complexity to calculate QK.
- Attention matrix $\dim\left(\text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)\right) = N \times N$
  - Storing the attention matrix for each head $\rightarrow O(N^2 h)$.
- If $N \gg d_k, h$, the time and space complexity is $O(N^2)$.
  - Scalability, resource consumption, adoption, etc.

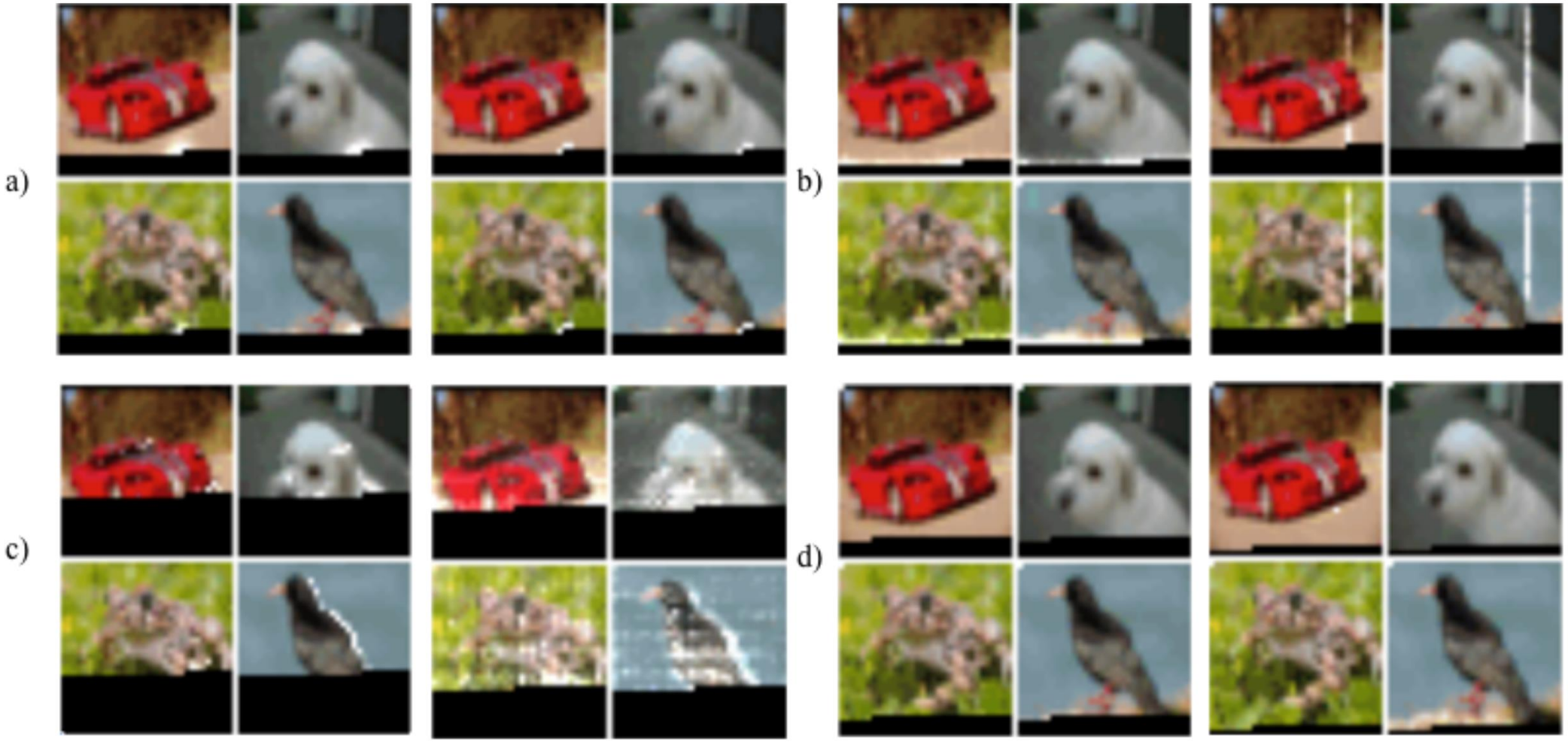
Solutions

- Sparse Transformers (April 2019) by Child et al.
  - $O\left(N\sqrt[p]{N}\right)$ time and space cost.
- Linformer (June 2020) by Wang et al.
- Linearized Attention (August 2020) by Katharopoulos et al.
- Longformer (December 2020) by Beltagy et al.
- Random Feature Attention (RFA) (March 2021) by Peng et al.
  - $O(N)$ time and space cost.
  - Some even constant space cost.

Sparse Transformers (Apr 2019)

Generating Long Sequences with Sparse Transformers
Rewon Child, Scott Gray, Alec Radford, and Ilya Sutskever

Sparse Transformers: Motivation



Sparse Transformers: Motivation

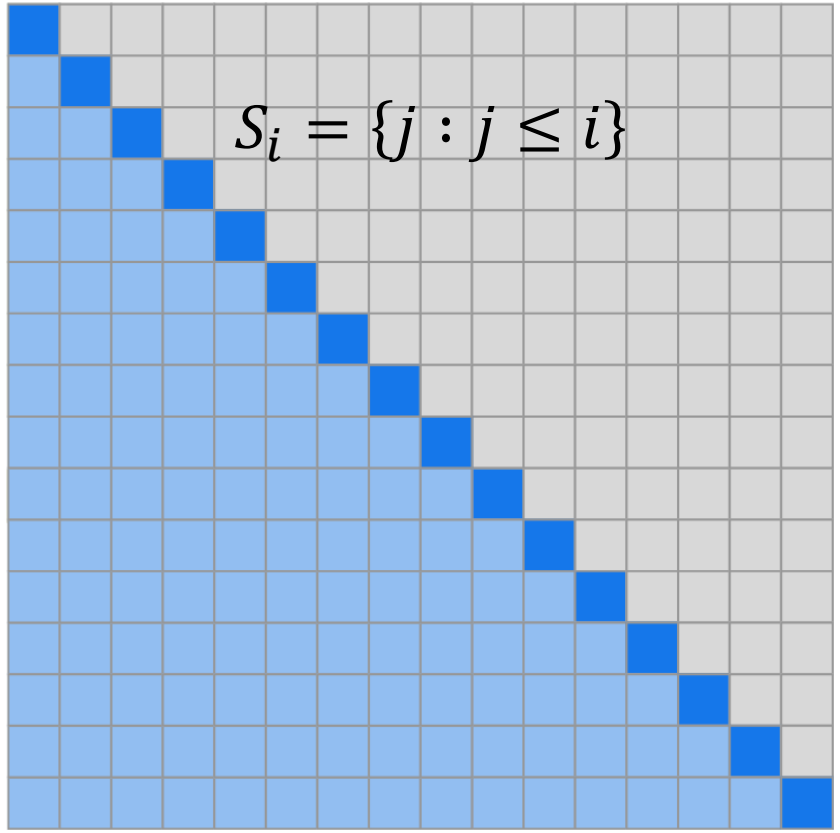
- Let $S = \{S_1, \dots, S_n\}$, $S_i :=$ input vectors that ith output vector attends.

$$\text{Attend}(X, S) = \left(a(x_i, S_i)\right)_{i \in \{1,\dots,n\}}$$

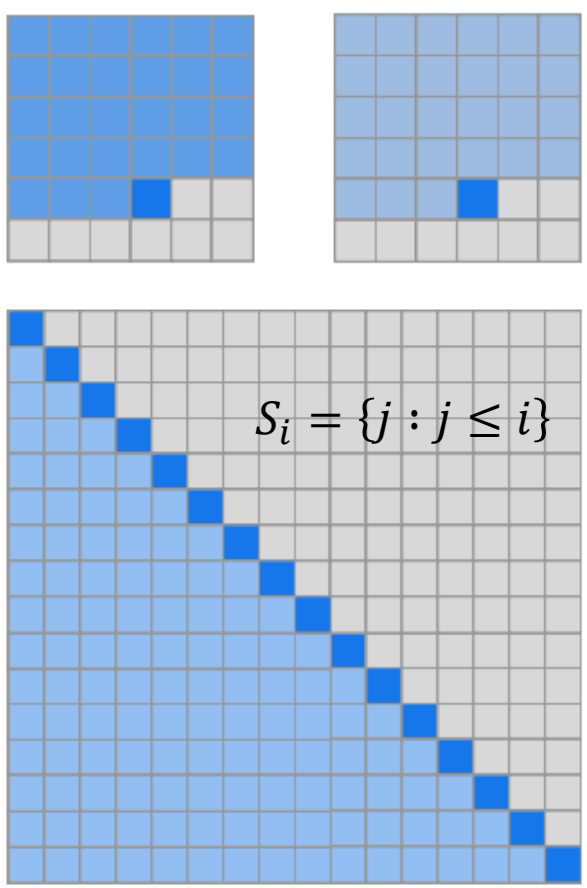
$$a(x_i, S_i) = \text{softmax}\left(\frac{(W_q x_i) K_{S_i}^T}{\sqrt{d}}\right) V_{S_i}$$

$$K_{S_i} = \left(W_k x_j\right)_{j \in S_i},\ V_{S_i} = \left(W_v x_j\right)_{j \in S_i}$$

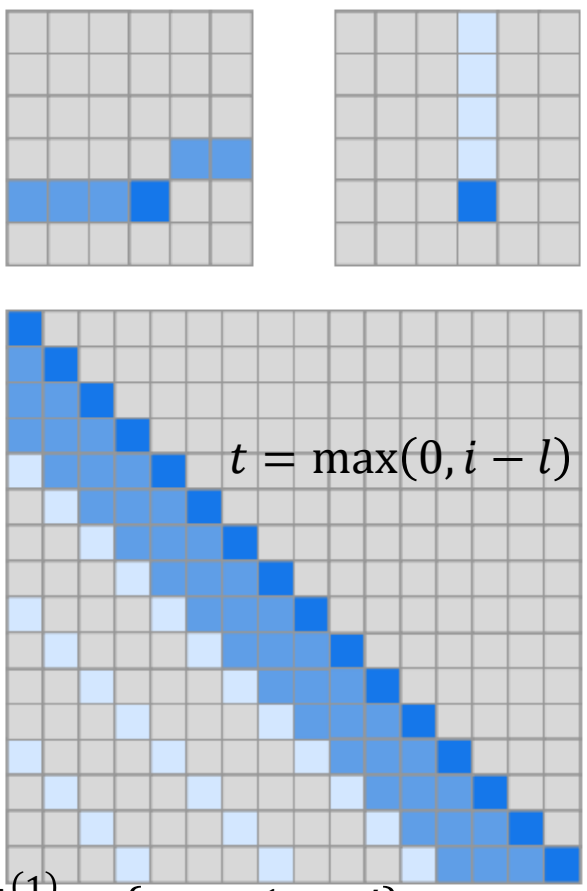
- p separate attention heads,
  Efficient $A_i^{(m)} \subset \{j : j \leq i\}$,
  $\left|A_i^{(m)}\right| \propto \sqrt[p]{n}$



Sparse Transformers: $p = 2$



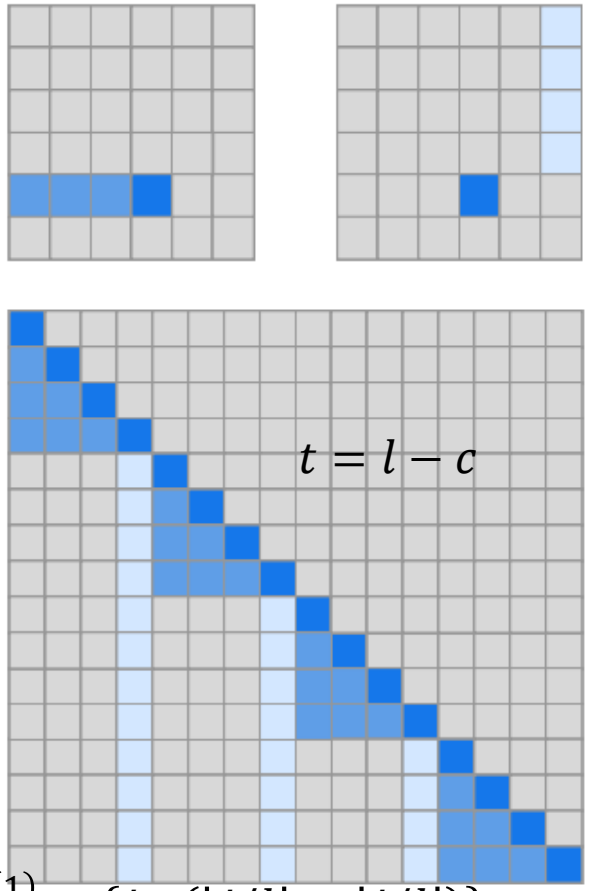
(a) Transformer



$A_i^{(1)} = \{t, t+1, \ldots, i\}$

$A_i^{(2)} = \{j : (i - j) \bmod l = 0\}$

(b) Sparse Transformer (strided)



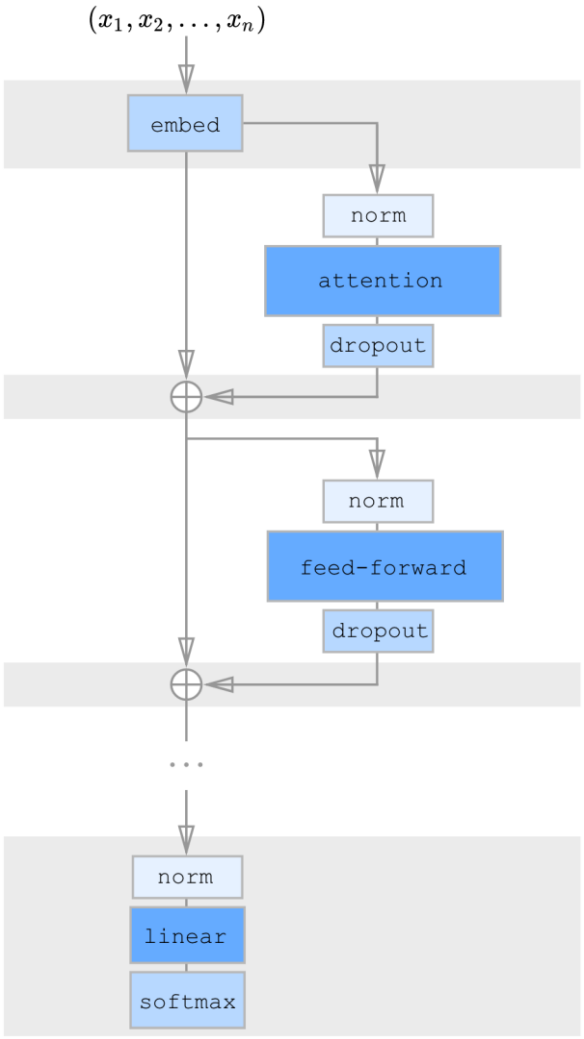
$A_i^{(1)} = \{j : (\lfloor j/l \rfloor = \lfloor i/l \rfloor)\}$

$A_i^{(2)} = \{j : j \bmod l \in \{t, t+1, \ldots, l\}\}$

(c) Sparse Transformer (fixed)

Sparse Transformers: Space Complexity

- Gradient Checkpoint & recomputation
- Only greyed blocks are stored
  - Attention and feed-forward recomputed



Sparse Transformers: Experiments

UNIMORE
UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

Table 1. Summary of our findings for density modeling tasks. Results are reported in bits per byte, which is equivalent to bits per dim for image tasks. M refers to millions of parameters.

Model	Bits per byte
CIFAR-10	
PixelCNN (Oord et al., 2016)	3.03
PixelCNN++ (Salimans et al., 2017)	2.92
Image Transformer (Parmar et al., 2018)	2.90
PixelSNAIL (Chen et al., 2017)	2.85
Sparse Transformer 59M (strided)	**2.80**
Enwik8	
Deeper Self-Attention (Al-Rfou et al., 2018)	1.06
Transformer-XL 88M (Dai et al., 2018)	1.03
Transformer-XL 277M (Dai et al., 2018)	**0.99**
Sparse Transformer 95M (fixed)	**0.99**
ImageNet 64x64	
PixelCNN (Oord et al., 2016)	3.57
Parallel Multiscale (Reed et al., 2017)	3.7
Glow (Kingma & Dhariwal, 2018)	3.81
SPN 150M (Menick & Kalchbrenner, 2018)	3.52
Sparse Transformer 152M (strided)	**3.44**
Classical music, 5 seconds at 12 kHz	
Sparse Transformer 152M (strided)	**1.97**

Table 2. Sparse patterns showed increased speed and also better loss on the datasets where we could compare both, which may point to a useful inductive bias in the patterns we learned or an underlying optimization issue with full attention.

Model	Bits per byte	Time/Iter
Enwik8 (12,288 context)		
Dense Attention	1.00	1.31
Sparse Transformer (Fixed)	**0.99**	0.55
Sparse Transformer (Strided)	1.13	0.35
CIFAR-10 (3,072 context)		
Dense Attention	2.82	0.54
Sparse Transformer (Fixed)	2.85	0.47
Sparse Transformer (Strided)	**2.80**	0.38

Table 3. We observe increased compression of Enwik8 with longer contexts, suggesting the Sparse Transformer can effectively incorporate long-term dependencies.

Minimum context length during evaluation	Bits per byte
6,144 tokens	0.9952
9,216 tokens	0.9936
10,752 tokens	0.9932
11,904 tokens	0.9930
12,096 tokens	0.9922
12,160 tokens	**0.9908**

Linformer (June 2020)

Linformer: Self-Attention with Linear Complexity
Sinong Wang, Belinda Z. Li, Madian Khabsa, Han Fang, and Hao Ma

Linformer: Idea

UNIMORE
UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

- Recall

$$\text{Attention}(Q_i, K_i, V_i) = \text{softmax}\left(\frac{Q_i K_i^T}{\sqrt{d_k}}\right) V_i$$

  where $\dim(K_i) = \dim(V_i) = N \times d_k$
- Project K_i and V_i with $E_i, F_i \in \mathbb{R}^{k \times n}$ into $\tilde{K}_i = E_i K_i, \tilde{V}_i = F_i V_i \in \mathbb{R}^{k \times d_k}$ such that $k \ll N$.
- Now softmax only requires $O(Nk)$ computational cost.
- The attention matrix is of dimension $N \times k$, thus storage takes $O(N \times k \times h)$ which is linear to N when $k, h \ll N$.

Linformer: Theorem

UNIMORE
UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

- Let $k = \frac{9\log(d_k)}{\varepsilon^2 - \varepsilon^3}, E_i = \delta R, F = e^{-\delta} R$, where $R \in \mathbb{R}^{n \times k}$ with i.i.d. entries from $N\left(0, \frac{1}{k}\right), \delta = \frac{1}{2^N}$, this theorem still holds, i.e.,

$$\text{softmax}\left(\frac{Q_i(E_i K_i)^T}{\sqrt{d_k}}\right) F_i V_i \sim \text{softmax}\left(\frac{Q_i {K_i}^T}{\sqrt{d_k}}\right) V_i$$

Theorem 2. *(Linear self-attention) For any* $Q_i, K_i, V_i \in \mathbb{R}^{n \times d}$ *and* $W_i^Q, W_i^K, W_i^V \in \mathbb{R}^{d \times d}$*, if* $k = \min\{\Theta(9d\log(d)/\epsilon^2), 5\Theta(\log(n)/\epsilon^2)\}$*, then there exists matrices* $E_i, F_i \in \mathbb{R}^{n \times k}$ *such that, for any row vector* w *of matrix* $QW_i^Q(KW_i^K)^T/\sqrt{d}$*, we have*

$$\Pr\left(\|\text{softmax}(wE_i^T)F_i V W_i^V - \text{softmax}(w)VW_i^V\| \leq \epsilon\|\text{softmax}(w)\|\|VW_i^V\|\right) > 1 - o(1) \quad (8)$$

Linformer: Experiments

- The performance aligns with that of the standard transformer.
- Different sharing:
  - Headwise: same $E_i = E$ and $F_i = F$ for all i.
  - Key-value: $E_i = F_i = E$.
  - Layerwise: single E for all.
- For different n and k: the performance merges eventually.

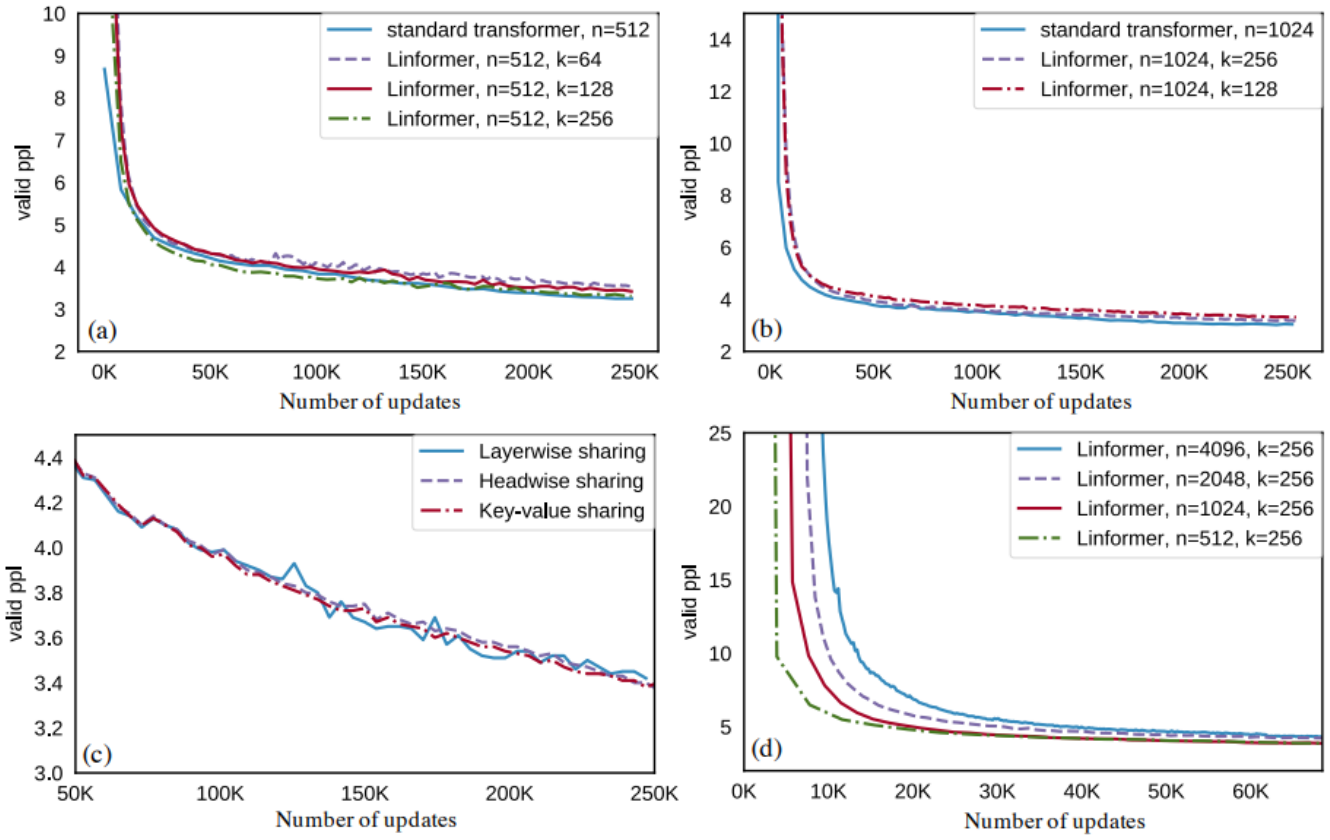


Figure 3: Pretraining validation perplexity versus number of updates.

Linformer: Experiments

UNIMORE
UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

n	Model	SST-2	IMDB	QNLI	QQP	Average
512	Liu et al. (2019), RoBERTa-base	93.1	94.1	90.9	**90.9**	92.25
	Linformer, 128	92.4	94.0	90.4	90.2	91.75
	Linformer, 128, shared kv	**93.4**	93.4	90.3	90.3	91.85
	Linformer, 128, shared kv, layer	93.2	93.8	90.1	90.2	91.83
	Linformer, 256	93.2	94.0	90.6	90.5	92.08
	Linformer, 256, shared kv	93.3	93.6	90.6	90.6	92.03
	Linformer, 256, shared kv, layer	93.1	94.1	**91.2**	90.8	**92.30**
512	Devlin et al. (2019), BERT-base	92.7	93.5	91.8	89.6	91.90
	Sanh et al. (2019), Distilled BERT	91.3	92.8	89.2	88.5	90.45
1024	Linformer, 256	93.0	93.8	90.4	90.4	91.90
	Linformer, 256, shared kv	93.0	93.6	90.3	90.4	91.83
	Linformer, 256, shared kv, layer	93.2	**94.2**	90.8	90.5	92.18

Table 2: Dev set results on benchmark natural language understanding tasks. The RoBERTa-base model here is pretrained with same corpus as BERT.

Linformer: Experiments

length n	projected dimensions k				
	128	256	512	1024	2048
512	1.5x	1.3x	-	-	-
1024	1.7x	1.6x	1.3x	-	-
2048	2.6x	2.4x	2.1x	1.3x	-
4096	3.4x	3.2x	2.8x	2.2x	1.3x
8192	5.5x	5.0x	4.4x	3.5x	2.1x
16384	8.6x	7.8x	7.0x	5.6x	3.3x
32768	13x	12x	11x	8.8x	5.0x
65536	20x	18x	16x	14x	7.9x

length n	projected dimensions k				
	128	256	512	1024	2048
512	1.7x	1.5x	-	-	-
1024	3.0x	2.9x	1.8x	-	-
2048	6.1x	5.6x	3.6x	2.0x	-
4096	14x	13x	8.3x	4.3x	2.3x
8192	28x	26x	17x	8.5x	4.5x
16384	56x	48x	32x	16x	8x
32768	56x	48x	36x	18x	16x
65536	60x	52x	40x	20x	18x

Table 3: Inference-time efficiency improvements of the Linformer over the Transformer, across various projected dimensions k and sequence lengths n. Left table shows time saved. Right table shows memory saved.

Linearized Attention (Aug 2020)

Transformers are RNNs: Fast Autoregressive Transformers with Linear Attention
Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret

Linearized Attention

- Recall the definition of the $\mathbf{softmax}$ function:

$$\mathrm{softmax}(x)_i = \frac{e^{x_i}}{\sum_{j=1}^{n} e^{x_i}} \text{ for } i = 1, \dots, n, x = (x_1, \dots, x_n).$$

- Thus, let M_i denote the ith row of matrix M, and let Y be the output of the attention mechanism, then

$$Y_i = \mathrm{softmax}\left(\frac{Q_i K}{\sqrt{d_k}}\right) V = \frac{\exp\left(\frac{Q_i K}{\sqrt{d_k}}\right) V}{\sum_{j=1}^{n} \exp\left(\frac{Q_i K_j}{\sqrt{d_k}}\right)} = \frac{\sum_{j=1}^{N} \mathrm{sim}(Q_i, K_j) V_j}{\sum_{j=1}^{N} \mathrm{sim}(Q_i, K_j)}$$

where $\mathrm{sim}(q, k) = \exp\left(\frac{q \cdot k}{\sqrt{d_k}}\right)$.

Linearized Attention

- The only constraint on $\mathbf{sim}(\cdot)$ is non-negativity.
  - Any $k(x, y) : \mathbb{R}^{2\times d_{model}} \rightarrow \mathbb{R}_+$ works.
- Let $\phi(x)$ be the *feature representation* of such a kernel. Then

$$Y_i = \frac{\sum_{j=1}^{N} \phi(Q_i)^T \phi(K_j) V_j}{\sum_{j=1}^{N} \phi(Q_i)^T \phi(K_j)} = \frac{\phi(Q_i)^T \sum_{j=1}^{N} \phi(K_j) V_j^T}{\phi(Q_i)^T \sum_{j=1}^{N} \phi(K_j)}$$

- $\sum_{j=1}^{N} \phi(K_j) V_j^T$ and $\sum_{j=1}^{N} \phi(K_j)$ can be computed once and reused
  - $O(N)$ time and space complexity
- $\phi(x) = \mathrm{elu}(x) + 1,\ \mathrm{elu}(x) = \begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$

Linearized Attention: Causal Masking

- Causal Masking: masking the attention computation such that the ith position can only be influenced by a position j if and only if $j \leq i$.

$$Y_i = \frac{\phi(Q_i)^T \sum_{j=1}^{i} \phi(K_j) V_j^T}{\phi(Q_i)^T \sum_{j=1}^{i} \phi(K_j)} = \frac{\phi(Q_i)^T S_i}{\phi(Q_i)^T Z_i}$$

where $S_i = \sum_{j=1}^{i} \phi(K_j) V_j^T$, $Z_i = \sum_{j=1}^{i} \phi(K_j)$.

- Notice that S_i, Z_i can be computed from S_{i-1}, Z_{i-1} in constant time.
- How about space complexity?

Linearized Attention: Causal Masking

- Given numerator $\overline{Y_i} = \phi(Q_i)^T \sum_{j=1}^{i} \phi(K_j) V_j^T$ and the gradient of loss fuction $\nabla_{\overline{Y_i}} \mathcal{L}$, they derived the gradients of Q, K, V

$$\nabla_{\phi(Q_i)} \mathcal{L} = \nabla_{\overline{Y_i}} \mathcal{L} \left(\sum_{j=1}^{i} \phi(K_j) V_j^T \right)^T$$

$$\nabla_{\phi(K_i)} \mathcal{L} = \left(\sum_{j=1}^{N} \phi(Q_j) V_j^T \left(\nabla_{\overline{Y_j}} \mathcal{L} \right)^T \right) V_i$$

$$\nabla_{V_i} \mathcal{L} = \left(\sum_{j=1}^{N} \phi(Q_j) \left(\nabla_{\overline{Y_j}} \mathcal{L} \right)^T \right)^T \phi(K_i)$$

Linearized Attention: RNN

- Transformers are RNNs.
- s and z are the hidden states.
- x_i is the ith input.
- y_i is the ith output.

$$
\begin{aligned}
s_0 &= 0, \\
z_0 &= 0, \\
s_i &= s_{i-1} + \phi\left(x_i W_K\right)\left(x_i W_V\right)^T, \\
z_i &= z_{i-1} + \phi\left(x_i W_K\right), \\
y_i &= f_l\left(\frac{\phi\left(x_i W_Q\right)^T s_i}{\phi\left(x_i W_Q\right)^T z_i} + x_i\right).
\end{aligned}
$$

Linearized Attention: Experiments

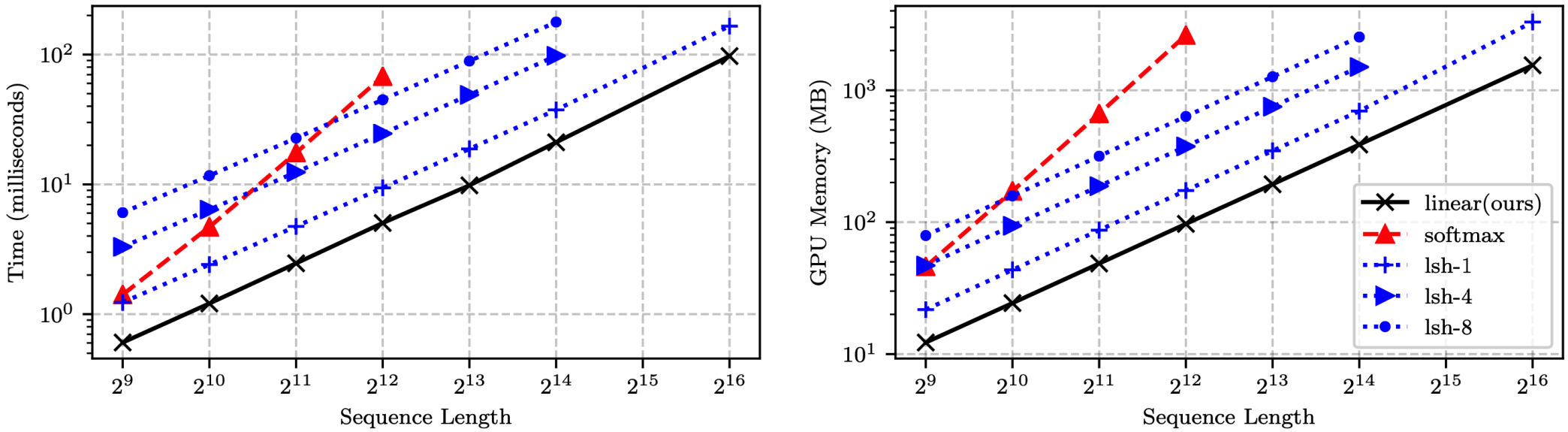


Figure 1: Comparison of the computational requirements for a forward/backward pass for Reformer (lsh-X), softmax attention and linear attention. Linear and Reformer models scale linearly with the sequence length unlike softmax which scales with the square of the sequence length both in memory and time. Full details of the experiment can be found in § 4.1.

Linearized Attention: Experiments

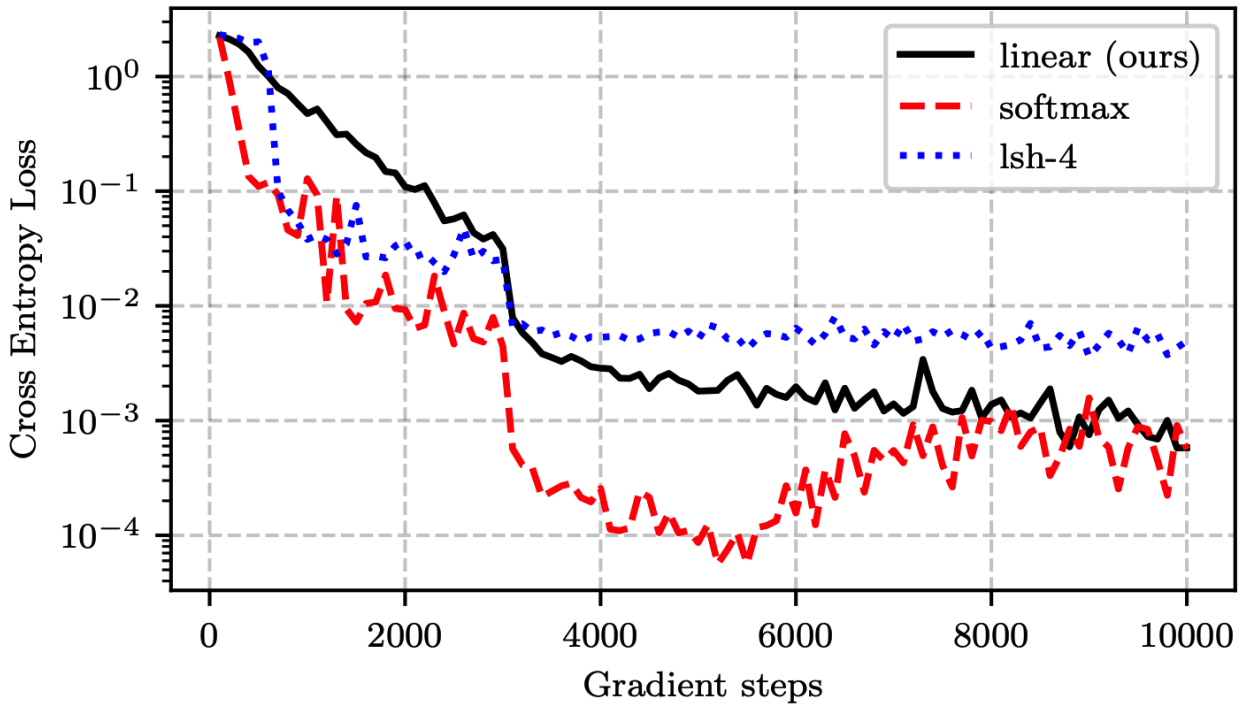


Figure 2: Convergence comparison of *softmax*, *linear* and *reformer* attention on a sequence duplication task. *linear* converges stably and reaches the same final performance as softmax. The details of the experiment are in § 4.1.

Linearized Attention: Experiments

UNIMORE
UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

Method	Bits/dim	Images/sec	
Softmax	0.621	0.45	(1×)
LSH-1	0.745	0.68	(1.5×)
LSH-4	0.676	0.27	(0.6×)
Linear (ours)	0.644	**142.8**	**(317×)**

Table 1: Comparison of autoregressive image generation of MNIST images. Our linear transformers achieve almost the same bits/dim as the full softmax attention but more than 300 times higher throughput in image generation. The full details of the experiment are in § 4.2.1.

Method	Bits/dim	Images/sec	
Softmax	3.47	0.004	(1×)
LSH-1	3.39	0.015	(3.75×)
LSH-4	3.51	0.005	(1.25×)
Linear (ours)	3.40	**17.85**	**(4,462×)**

Table 2: We train autoregressive transformers for 1 week on a single GPU to generate CIFAR-10 images. Our linear transformer completes 3 times more epochs than softmax, which results in better perplexity. Our model generates images 4,000× faster than the baselines. The full details of the experiment are in § 4.2.2.

Linearized Attention: Experiments

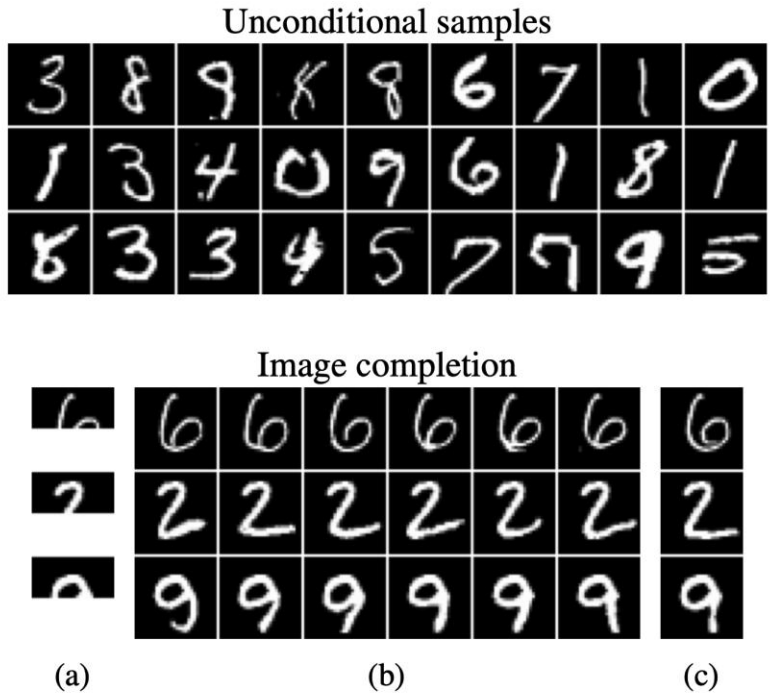


Figure 3: Unconditional samples and image completions generated by our method for MNIST. (a) depicts the occluded orignal images, (b) the completions and (c) the original. Our model achieves comparable bits/dimension to softmax, while having more than **300 times** higher throughput, generating **142 images/second**. For details see § 4.2.1.

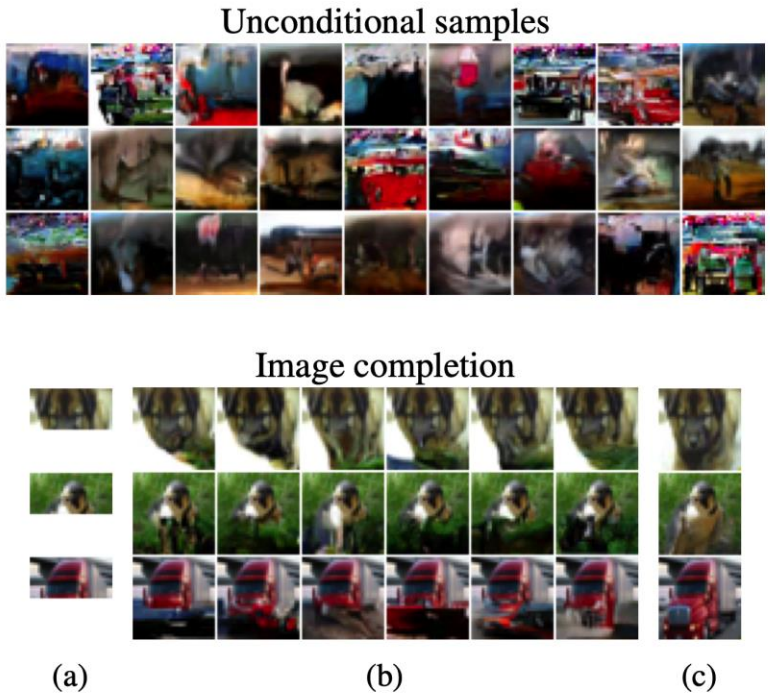


Figure 4: Unconditional samples and image completions generated by our method for CIFAR-10. (a) depicts the occluded orignal images, (b) the completions and (c) the original. As the sequence length grows linear transformers become more efficient compared to softmax attention. Our model achieves more than **4,000 times** higher throughput and generates **17.85 images/second**. For details see § 4.2.2.

Linearized Attention: Experiments

Method	Validation PER	Time/epoch (s)
Bi-LSTM	10.94	1047
Softmax	5.12	2711
LSH-4	9.33	2250
Linear (ours)	8.08	**824**

Table 3: Performance comparison in automatic speech recognition on the WSJ dataset. The results are given in the form of phoneme error rate (PER) and training time per epoch. Our model outperforms the LSTM and Reformer while being faster to train and evaluate. Details of the experiment can be found in § 4.3.

Longformer (Dec 2020)

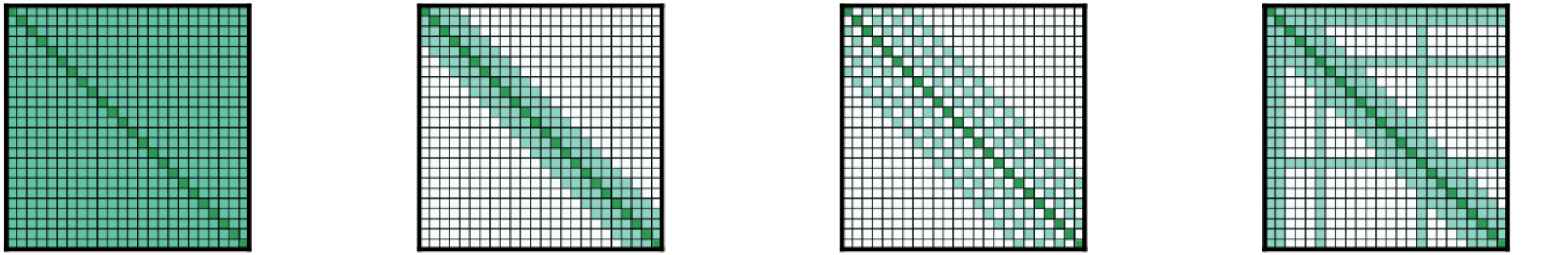
Longformer: The Long-Document Transformer
Iz Beltagy, Matthew E. Peters, and Arman Cohan

Longformer: Idea

- **Local context** is important.
- ***Sparsify*** the full self-attention matrix
  - "attention pattern" specify pairs of input locations attending to one another.
- If the window is fixed (i.e. independent of N), the computational and space cost is $O(N)$.

Longformer

- Sliding Window: give fixed window size w, each token attends to $\frac{1}{2}w$ tokens on each side.
- Dilated Sliding Window: with gaps of size dilation d.
- Global Attention: Specific tokens that receive global attention.
- They can all achieve $O(N)$ time and space complexity.



(a) Full n^2 attention (b) Sliding window attention (c) Dilated sliding window (d) Global+sliding window

Figure 2: Comparing the full self-attention pattern and the configuration of attention patterns in our Longformer.

Longformer: Bits-per-Character

Model	#Param	Dev	Test
Dataset `text8`			
T12 (Al-Rfou et al., 2018)	44M	-	1.18
Adaptive (Sukhbaatar et al., 2019)	38M	1.05	1.11
BP-Transformer (Ye et al., 2019)	39M	-	1.11
Our Longformer	41M	1.04	**1.10**
Dataset `enwik8`			
T12 (Al-Rfou et al., 2018)	44M	-	1.11
Transformer-XL (Dai et al., 2019)	41M	-	1.06
Reformer (Kitaev et al., 2020)	-	-	1.05
Adaptive (Sukhbaatar et al., 2019)	39M	1.04	1.02
BP-Transformer (Ye et al., 2019)	38M	-	1.02
Our Longformer	41M	1.02	**1.00**

Table 2: *Small* model BPC on `text8` & `enwik8`

Model	#Param	Test BPC
Transformer-XL (18 layers)	88M	1.03
Sparse (Child et al., 2019)	≈100M	0.99
Transformer-XL (24 layers)	277M	0.99
Adaptive (Sukhbaatar et al., 2019)	209M	0.98
Compressive (Rae et al., 2020)	277M	0.97
Routing (Roy et al., 2020)	≈223M	0.99
Our Longformer	102M	0.99

Table 3: Performance of *large* models on `enwik8`

Model	Dev BPC
Decreasing w (from 512 to 32)	1.24
Fixed w (= 230)	1.23
Increasing w (from 32 to 512)	**1.21**
No Dilation	1.21
Dilation on 2 heads	**1.20**

Table 4: Top: changing window size across layers. Bottom: with/without dilation (@ 150K steps on phase1)

Model	base	large
RoBERTa (seqlen: 512)	1.846	1.496
Longformer (seqlen: 4,096)	10.299	8.738
+ copy position embeddings	1.957	1.597
+ 2K gradient updates	1.753	1.414
+ 65K gradient updates	1.705	1.358
Longformer (train extra pos. embed. only)	1.850	1.504

Table 5: MLM BPC for RoBERTa and various pretrained Longformer configurations.

Longformer: Experiments

UNIMORE
UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

	QA			Coref.	Classification	
Model	WikiHop	TriviaQA	HotpotQA	OntoNotes	IMDB	Hyperpartisan
RoBERTa-base	72.4	74.3	63.5	78.4	95.3	87.4
Longformer-base	**75.0**	**75.2**	**64.4**	**78.6**	**95.7**	**94.8**

Table 7: Summary of finetuning results on QA, coreference resolution, and document classification. Results are on the development sets comparing our Longformer-base with RoBERTa-base. TriviaQA, Hyperpartisan metrics are F1, WikiHop and IMDB use accuracy, HotpotQA is joint F1, OntoNotes is average F1.

Model	WikiHop	TriviaQA	HotpotQA
Current* SOTA	78.3	73.3	**74.2**
Longformer-large	**81.9**	**77.3**	73.2

Table 8: Leaderboard results of Longformer-large at time of submission (May 2020). All numbers are F1 scores.

Longformer: Experiments

Model	ans.	supp.	joint
TAP 2 (ensemble) (Glaß et al., 2019)	79.8	86.7	70.7
SAE (Tu et al., 2019)	79.6	86.7	71.4
Quark (dev) (Groeneveld et al., 2020)	81.2	87.0	72.3
C2F Reader (Shao et al., 2020)	81.2	87.6	72.8
Longformer-large	81.3	88.3	73.2
ETC-large† (Ainslie et al., 2020)	81.2	89.1	73.6
GSAN-large†	81.6	88.7	73.9
HGN-large (Fang et al., 2020)	82.2	88.5	74.2

Table 9: HotpotQA results in distractor setting test set. Quark's test results are not available. All numbers are F1 scores. † shows contemporaneous leaderboard submissions.

Model	Accuracy / Δ
Longformer (seqlen: 4,096)	73.8
RoBERTa-base (seqlen: 512)	72.4 / -1.4
Longformer (seqlen: 4,096, 15 epochs)	75.0 / +1.2
Longformer (seqlen: 512, attention: n^2)	71.7 / -2.1
Longformer (seqlen: 2,048)	73.1 / -0.7
Longformer (no MLM pretraining)	73.2 / -0.6
Longformer (no linear proj.)	72.2 / -1.6
Longformer (no linear proj. no global atten.)	65.5 / -8.3
Longformer (pretrain extra position embed. only)	73.5 / -0.3

Table 10: WikiHop development set ablations

	R-1	R-2	R-L
Discourse-aware (2018)	35.80	11.05	31.80
Extr-Abst-TLM (2020)	41.62	14.69	38.03
Dancer (2020)	42.70	16.54	38.44
Pegasus (2020)	44.21	16.95	38.83
LED-large (seqlen: 4,096) (ours)	44.40	17.94	39.76
BigBird (seqlen: 4,096) (2020)	**46.63**	19.02	41.77
LED-large (seqlen: 16,384) (ours)	**46.63**	**19.62**	**41.83**

Table 11: Summarization results of Longformer-Encoder-Decoder (LED) on the arXiv dataset. Metrics from left to right are ROUGE-1, ROUGE-2 and ROUGE-L.

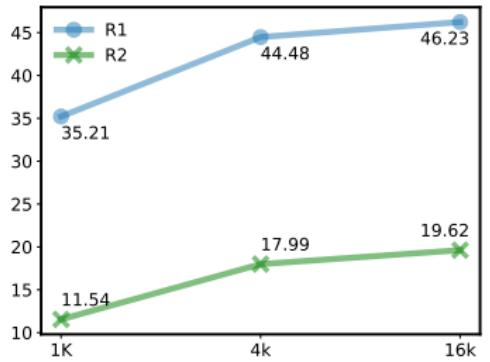


Figure 3: ROUGE-1 and ROUGE-2 of LED when varying the input size (arXiv validation set).